CS-4032: Web Programming

Assignment#02 - 2D Cricket Web Application (Web Development)

Question

Design and develop a **2D Cricket Web Application** similar to a simple batting game (as shown in the reference image). The application should be built using **HTML, CSS, Canvas or DOM-based rendering, JavaScript and React**. It must simulate cricket gameplay using a probability-based power bar system instead of pure randomness.

The objective is to apply web development concepts such as UI design, event handling, animations, structured programming, and probability-based game logic.

Problem Statement

Create a **single-player 2D cricket batting game** where the user plays as a batsman. On each ball, the user selects a batting style (Aggressive or Defensive), and a probability-based power bar determines the outcome.

The match must have fixed overs (e.g. 2 overs = 12 balls) and limited wickets. The game ends when the allotted overs are completed or all wickets are lost.

Functional Requirements

Your 2D Cricket Web App must include the following:

1. A 2D cricket ground/pitch layout displayed on the screen.
2. A batsman/bat and ball image or sprite visible on the pitch.
3. A scoreboard showing:
    - Total runs
    - Wickets fallen
    - Overs / balls remaining
4. Batting style selection (Aggressive and Defensive).
    - Aggressive style must have higher risk (higher wicket probability) and higher reward (higher boundary probability).
    - Defensive style must have lower risk and lower boundary probability.
5. A Probability-Based Power Bar system.
    - Each outcome (0, 1, 2, 3, 4, 6, and Wicket) must have an assigned probability.
    - For each batting style, the total probability must sum exactly to 1.
    - The power bar must visually represent probabilities proportionally.
    - If wicket = 0.40, then 40% of the power bar must represent the wicket region.
6. A moving slider that continuously travels across the power bar.
    - When the user clicks to play the shot, the slider position determines the outcome.
    - The result must depend strictly on which probability segment the slider is inside.
    - Random selection of outcome is not allowed.
7. Bowling animation where the ball visibly travels toward the batsman before each shot.
8. Batting animation triggered when the user plays the shot.
9. Proper game progression logic:
    - Fixed overs (2 overs = 12 balls).
    - Limited wickets (e.g., 2 wickets).
    - Game ends when overs finish or all wickets are lost.
10. A Restart Game option that resets:
    - Runs
    - Wickets
    - Overs
    - Batting style state

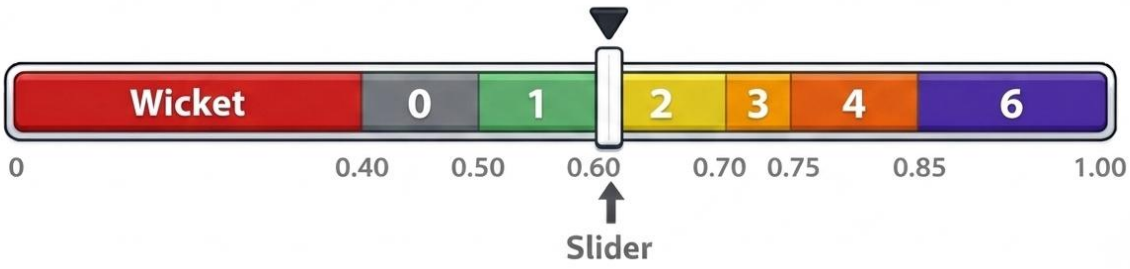
Example: How the Power Bar and Slider Work

To understand the probability-based power bar, consider the following example.

Suppose the Aggressive batting style uses the following probabilities:

Outcome	Probability
Wicket	0.40
0 Runs	0.10
1 Run	0.10
2 Runs	0.10
3 Runs	0.05
4 Runs	0.10
6 Runs	0.15
Total	**1.00**

The power bar must be divided into segments according to these probabilities. Each region must occupy the same proportion of the power bar as its probability.



The slider continuously moves across the power bar from left to right. When the user clicks to play the shot, the current position of the slider determines the result. For example, if the slider stops at 0.32, it lies within the Wicket region (0-0.40), therefore the result of the shot will be Wicket.

Therefore, the outcome is determined entirely by which segment of the power bar the slider is in when the player clicks.

Different batting styles must use different probability distributions, which will change the sizes of the power bar segments.

Game Rules

- Total overs: 2 (12 balls)
- Total wickets: 2 (or reasonable limit)
- Each batting attempt represents one ball.

UI / Design Requirements

- Use **2D graphics or images** (no 3D required).
- Layout should resemble a cricket field.
- Scoreboard must be clearly visible and updated dynamically.
- Power bar must clearly show different colored probability zones.
- Buttons should be user-friendly and responsive.
- Animations must be smooth and synchronized with gameplay.
- Mobile responsiveness is preferred.

Technologies Allowed

- HTML, CSS, JavaScript
- React
- Canvas or DOM-based rendering

Bonus Features (Optional)

- Dynamic Commentary system: Store multiple hardcoded commentary statements for each outcome (e.g., at least 3 for wicket, 3 for boundary). After each ball, randomly select and display one statement based on the result.
- Improved animations, sound effects, or UI enhancements.

Submission Instructions (ZIP Folder + PDF + GitHub Repository)

1. GitHub Repository
    - Submit the GitHub repository link containing your project code.
    - Your repository must include meaningful and regular commits showing development progress.
    - Repositories with very few commits or commits added only at the end may result in deduction of a
2. Source code (ZIP Folder)
    - Upload all project source code inside a ZIP folder.
    - Do not include the PDF report inside the ZIP folder.
3. PDF report containing:
    - Full-screen screenshots of gameplay, showing:
        1. Aggressive batting in action
        2. Defensive batting in action
        3. Power bar visible during gameplay
        4. Scoreboard updates reflecting game progress
        5. Game over screen

        Each screenshot must include a visible sticky note displaying your Name, Roll Number, and Section.

    - Brief explanation describing:
        1. Game Logic and working mechanism
        2. Mapping of probabilities to the power bar
        3. Implementation of animations

Naming Convention: XXi-XXXX-Section

Evaluation Criteria (100 Marks)

Note: Marks will be awarded solely based on the **viva conducted during the demo**.

Category	Component	Criteria Summary	Marks
Game Logic & Probability (30)	Aggressive Batting	Proper probability distribution, sums to 1, high-risk/high-reward logic, correct power bar mapping	10
	Defensive Batting	Proper probability distribution, sums to 1, lower-risk/lower-reward logic, correct power bar mapping	10
	Power Bar Logic	Segments proportional to probabilities, smooth slider, accurate click detection, correct outcome mapping	10
UI / 2D Design & Animation (30)	Field & Sprites	Cricket-like layout, batsman and ball visible, proper alignment	5
	Scoreboard	Displays runs, wickets, overs/balls, updates dynamically	5
	Batting Animation	Triggered correctly, smooth, synchronized with logic	7.5
	Bowling Animation	Ball visibly travels, consistent timing, resets correctly	7.5
	Power Bar Design	Clear segmentation, color distinction, visible slider, no glitches	5
JavaScript / React (20)	Modular Code	Separate functions, no duplication, logical organization, clean separation of concerns	10
	State Handling	Proper tracking of runs/wickets/balls, correct updates, proper use of variables or React hooks	10
Code Quality (10)	Comments, Naming & GitHub Repository	Meaningful comments, probability explanation, clear variable/function names, consistent style, **GitHub repo with meaningful and regular commits showing development progress**	10
Documentation (10)	PDF Submission	Required screenshots (with sticky note), explanation of probability mapping & animations	10
Bonus (Up to 10)	Commentary & Enhancements	Context-based commentary, multiple messages per outcome, optional improvements (configurable overs, UI polish, etc.)	+10

Note: Plagiarism is strictly prohibited. All work must be original.